

Reporte de Pruebas de Seguridad

Mariscos El Gordo — Sistema de Inventario

27 de septiembre de 2026 · Alcance: backend (Express + JWT) y frontend

1. Análisis de dependencias (npm audit)

Comando ejecutado:

```
npm audit
```

Resultado: 0 vulnerabilidades en las 372 dependencias del proyecto (85 de producción, 288 de desarrollo). Reporte completo en npm-audit.json, incluido en esta misma carpeta del repositorio.

```
found 0 vulnerabilities
```

Esto cubre vulnerabilidades conocidas y reportadas públicamente (CVEs) en paquetes de terceros como Express, jsonwebtoken, bcryptjs y helmet. Se recomienda volver a correr este comando periódicamente, ya que nuevas vulnerabilidades se descubren con el tiempo en dependencias existentes.

2. Revisión manual contra OWASP Top 10 (2021)

Revisión del código propio (no de dependencias) contra las 10 categorías de riesgo de OWASP.

#	Categoría	Estado	Detalle
A01	Broken Access Control	Cubierto	Middleware authenticate + authorize(rol) en todas las rutas protegidas. Los encargados solo pueden ver/modificar insumos, solicitudes y transferencias de su propia sucursal, validado en el controlador (no solo en el frontend).
A02	Cryptographic Failures	Cubierto	Contraseñas hasheadas con bcrypt (10 rondas). JWT firmado con clave secreta desde variable de entorno, expiración de 2 horas.
A03	Injection	Corregido	Ver Hallazgo #1 (XSS almacenado) más abajo. No aplica inyección SQL: no hay base de datos SQL.
A04	Insecure Design	Cubierto	Modelo de permisos por rol y por sucursal diseñado desde el inicio; mínimo privilegio: un encargado nunca ve datos de otra sucursal ni de CEDIS.
A05	Security Misconfiguration	Cubierto	Cabeceras de seguridad HTTP con helmet, incluida una Content-Security-Policy explícita. Sin stack traces expuestos al cliente.
A06	Vulnerable/Outdated Components	Cubierto	npm audit limpio (sección 1). Recomendado repetirlo antes de cada entrega o despliegue.
A07	Auth. Failures	Corregido	Ver Hallazgo #2 (fuerza bruta / costo de hash) más abajo.
A08	Software/Data Integrity	Cubierto	El pipeline de CI usa npm ci (instala exactamente lo fijado en package-lock.json).
A09	Logging/Monitoring	No implementado	Proyecto académico sin logging centralizado. Recomendado como mejora futura.

#	Categoría	Estado	Detalle
A10	SSRF	No aplica	El servidor no hace peticiones salientes a URLs proporcionadas por el usuario.

3. Hallazgos y correcciones aplicadas

Hallazgo #1 — XSS almacenado (Cross-Site Scripting) — Corregido

Severidad: Alta.

El frontend (public/app.js) insertaba datos guardados por el usuario (nombre de insumo, sucursal, usuario que solicitó/transfirió) directamente dentro de innerHTML al construir las tablas de Inventario, Solicitudes y Transferencias, sin escapar caracteres HTML.

Esto era especialmente grave porque el campo "nombre" de una solicitud lo escribe libremente cualquier encargado de sucursal (rol de menor privilegio), y ese texto se muestra tal cual en la tabla de Solicitudes que ve el admin: un encargado podía inyectar script que se ejecutara en la sesión del administrador — escalamiento de privilegios vía XSS.

Prueba de concepto realizada: se inició sesión como encargado_centro y se envió una solicitud con:

```
{ "categoria": "proteinas", "nombre": "<img src=x onerror=alert(document.cookie)>", "cantidadKg": 5 }
```

El servidor la aceptó y almacenó tal cual (201 Created), confirmando que se habría renderizado sin escapar en la sesión del administrador.

Corrección: se agregó una función escapeHtml() en public/app.js y se aplicó a todo campo de texto proveniente de datos guardados (nombre, sucursal, categoría, usuario que solicitó/transfirió) antes de insertarlo en innerHTML, en las tres tablas.

Hallazgo #2 — Sin límite de intentos de login + costo de hash bajo — Corregido

Severidad: Media.

El endpoint POST /api/auth/login no tenía límite de intentos, permitiendo un ataque de fuerza bruta ilimitado contra contraseñas de usuarios. Además, el costo de bcrypt estaba en 8 rondas, por debajo de las 10 recomendadas actualmente como mínimo.

Corrección:

- Se agregó express-rate-limit al login: máximo 10 intentos por IP cada 15 minutos (respuesta 429 al exceder). Verificado manualmente: los intentos 11 y 12 seguidos regresan 429 Too Many Requests.
- Se subió el costo de bcrypt de 8 a 10 rondas, tanto para el registro de nuevos usuarios como para los usuarios sembrados de prueba.

4. Análisis de calidad de código y seguridad adicional (SonarCloud)

Se conectó el repositorio a SonarCloud (análisis vía GitHub Actions). Resultado del primer análisis:

Métrica	Resultado	Calificación
Líneas de código	2.5k	—
Security (código propio)	4 issues abiertos	C
Reliability (bugs)	0 issues abiertos	A
Maintainability (code smells)	9 issues abiertos	A

Hallazgos de seguridad en el pipeline CI/CD (identificados, no corregidos)

Los 4 issues de "Security" que reporta SonarCloud no están en el código de la aplicación (backend/frontend), sino en el archivo del pipeline .github/workflows/ci-cd.yml. Se documentan aquí como hallazgos identificados; se decidió no aplicar la corrección en esta entrega para no alterar el pipeline ya validado, quedando como recomendación de mejora.

Hallazgo A — Dependencias de GitHub Actions referenciadas por tag y no por commit SHA (Severidad: Alta, x2)

Categoría OWASP relacionada: A08 (Software and Data Integrity Failures).

El pipeline usa acciones de terceros por versión (ej. actions/checkout@v4, SonarSource/sonarcloud-github-action@master) en lugar del hash exacto del commit. Un tag es mutable: si la acción fuera comprometida y el mantenedor (o un atacante) moviera el tag a un commit malicioso, el pipeline ejecutaría ese código automáticamente en el siguiente push, sin que nadie lo note.

Ubicaciones señaladas por SonarCloud: .github/workflows/ci-cd.yml líneas 25 y 60 (actions/checkout@v4 y SonarSource/sonarcloud-github-action@master, esta última especialmente riesgosa por apuntar a una rama, no a un tag).

Commits SHA exactos ya identificados por si se aplica la corrección después:

- actions/checkout@v4 → 11d5960a326750d5838078e36cf38b85af677262
- actions/setup-node@v4 → 49933ea5288caeca8642d1e84afbd3f7d6820020
- actions/upload-artifact@v4 → ea165f8d65b6e75b540449e92b4886f43607fa02
- zaproxy/action-baseline@v0.12.0 → 66042c8e7e24680119199a017e5b0e8603bf4dae
- SonarSource/sonarcloud-github-action@master → debería fijarse a un release estable (v5.0.0 → ffc3010689be73b8e5ae0c57ce35968afd7909e8) en vez de a la rama master

Hallazgo B — Instalación de dependencias sin --ignore-scripts (Severidad: Media, x2)

Categoría OWASP relacionada: A08 (Software and Data Integrity Failures).

Los pasos npm ci del pipeline no usan la bandera --ignore-scripts. Sin ella, si alguna dependencia (propia o transitiva) trajera un script malicioso de instalación (preinstall/postinstall), se ejecutaría automáticamente en el runner de GitHub Actions. Ninguna de las dependencias actuales del proyecto requiere scripts de instalación (son paquetes JavaScript puros), por lo que agregar --ignore-scripts sería seguro, pero se deja pendiente.

Ubicaciones señaladas por SonarCloud: .github/workflows/ci-cd.yml líneas 25 y 56 (los dos pasos npm ci).

Nota sobre Coverage y Duplications

El primer análisis de SonarCloud se disparó de forma automática (zero-config) antes de correr la suite de pruebas, por lo que no reflejó la cobertura real del proyecto. El dato de cobertura correcto (94%+, 64 pruebas) es el que se documenta en tests/coverage-report.txt, generado con Jest directamente.

5. Resumen

Métrica	Resultado
Vulnerabilidades en dependencias (npm audit)	0
Vulnerabilidades encontradas en código de la app (revisión manual OWASP)	2 (XSS almacenado, fuerza bruta/hash débil)
Vulnerabilidades corregidas en código de la app	2 de 2
Hallazgos de seguridad en el pipeline CI/CD (SonarCloud)	2 (4 issues): SHA sin fijar, instalación sin --ignore-scripts

Métrica	Resultado
Hallazgos del pipeline corregidos	0 de 2 — documentados como mejora futura (decisión del equipo)
Bugs de confiabilidad (SonarCloud)	0
Code smells de mantenibilidad (SonarCloud)	9 (calificación A)
Pruebas unitarias tras la corrección	64/64 pasando, 94%+ de cobertura

Todas las correcciones aplicadas están incluidas en el historial de commits del repositorio y no rompieron ninguna prueba existente. Los hallazgos del pipeline CI/CD quedaron identificados y documentados, con la corrección propuesta lista para aplicarse en una siguiente iteración.